SHOPEASE

E-Commerce Website Development

INTERNSHIP PROJECT REPORT

Submitted as part of the Paid Internship Program

Organization

Corizo Edutech Private Limited

Project Team

1. **VENKATA SHARTH KUMAR JAKKI**
2. **SHRATI TIRUMALESHWAR HEDGE**
3. **SVVVS. BHASKAR**
4. **PRAADEEP S**
5. **DHANVI S**

Internship Project

2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Corizo Edutech Private Limited** for providing us with the opportunity to work on the ShopEase E-Commerce Website project as part of our internship.

This internship has helped us gain practical exposure to web development, project planning, frontend design, backend integration, database-related operations, version control, and collaborative software development. Working on ShopEase gave us an opportunity to apply the concepts learned during our academic studies to a practical project.

We are thankful to our mentors and the internship team for their guidance, support, and valuable suggestions throughout the project. Their guidance helped us understand the importance of writing maintainable code, resolving technical issues, testing application features, and following a structured development process.

We would also like to thank our team members for their cooperation, shared learning, and contributions toward the development and documentation of ShopEase. Working together helped us improve our communication, problem-solving, and teamwork skills.

Finally, we express our gratitude to everyone who directly or indirectly supported us during the internship and contributed to our learning experience.

Team Members

1. VENKATA SHARTH KUMAR JAKKI
2. SHRATI TIRUMALESHWAR HEDGE
3. SVVVS. BHASKAR
4. PRAADEEP S
5. DHANVI S

TABLE OF CONTENT

1. INTRODUCTION 1 - 2
1.1 BACKGROUND 1
1.2 PROJECT OVERVIEW 1
1.3 PROBLEM STATEMENT 1
1.4 SCOPE OF THE PROJECT 1 - 2
2. PROJECT OBJECTIVES 2
2.1 PRIMARY OBJECTIVE 2
2.2 SPECIFIC OBJECTIVES 2
3. SYSTEM REQUIREMENT AND ANALYSIS 2 - 6
3.1 HARDWARE REQUIREMENTS 3
3.2 SOFTWARE REQUIREMENTS 3
3.3 FUNCTIONAL REQUIREMENTS 3
3.3.1 USER MODULE 3 - 4
3.3.2 PRODUCT MODULE 4
3.3.3 ADMIN MODULE 4
3.3.4 BACKEND MODULE 4 - 5
3.4 NON-FUNCTIONAL REQUIREMENTS 5
3.5 USER ROLES 6
3.6 SYSTEM ANALYSIS 6
4. SYSTEM DESIGN AND ARCHITECTURE 6 - 9
4.1 SYSTEM ARCHITECTURE 7
4.2 SYSTEM MODULES 7- 8
4.3 DATA FLOW 8
4.4 DATABASE DESIGN 8
4.5 WEBSITE DIRECTORY STRUCTURE 8 - 9
4.6 ARCHITECTURE SUMMARY 9
5. TECHNOLOGIES USED 9 - 12
5.1 HTML 9
5.2 CSS 9 - 10
5.3 JAVASCRIPT 10
5.4 PYTHON 10

5.5 Flask Framework 10 -.....11
5.6 Database Technologies.....11
5.7 Flask-CORS.....11
5.8 Python Dotenv.....11
5.9 Gunicorn.....11
5.10 Git and GitHub.....11
5.11 Summary of Technologies.....12
6. SYSTEM IMPLEMENTATION.....12 - 17
6.1 Frontend Implementation.....12 - 13
6.2 User Registration and Login Module.....13 - 14
6.3 Product Browsing Module.....14
6.4 Shopping Cart Module.....14 - 15
6.5 Order Management Module.....15
6.6 Admin Login Module.....15
6.7 Admin Dashboard Module.....15 - 16
6.8 Product Management Module.....16
6.9 Customer Management Module.....16
6.10 Backend API Implementation.....16 - 17
6.11 Implementation Summary.....17
7. TESTING AND RESULTS.....17 - 19
7.1 Objectives of Testing.....17
7.2 Testing Methodology.....17 - 18
7.3 Test Cases.....18
7.4 Test Results.....18 - 19
7.5 Result Analysis.....19
7.6 Limitations of the Current Evaluation.....19
8. TEAM CONTRIBUTIONS.....19 - 20
8.1 Team Member Responsibilities.....19
8.2 Collaborative Development.....19
8.3 Importance of Teamwork.....19 - 20
9. CONCLUSION.....20 - 21
10. FUTURE ENHANCEMENTS.....21 - 22
10.1 Online Payment Integration.....21
10.2 Product Search and Filtering.....21

10.3 IMPROVED AUTHENTICATION AND SECURITY 21 - 22
10.4 ORDER TRACKING 22
10.5 PRODUCT REVIEWS AND RATINGS 22
10.6 RESPONSIVE USER INTERFACE 22
10.7 INVENTORY MANAGEMENT 22
10.8 NOTIFICATIONS 22
10.9 PERFORMANCE AND SCALABILITY 22
10.10 REPORTING AND ANALYTICS 22
10.11 SUMMARY 22
11. REFERENCES ..... 22 - 23

List of Figures

Figure No.	Figure Description	Page No
Figure. 4.1	System Architecture Diagram	13
Figure. 6.1	ShopEase Home Page	18
Figure. 6.2.1	User Registration Page	19
Figure. 6.2.2	User Login Page	19
Figure. 6.3	Product Listing Page	20
Figure. 6.4	Shopping Cart Page	20
Figure. 6.7	Admin Dashboard	21

List of Tables

Figure No.	Figure Description	Page No
Table. 3.1	Hardware Requirements	9
Table. 3.2	Software Requirements	9
Table. 3.3	Functional Requirements	10 – 11
Table. 3.4	Non-Functional Requirements	11
Table. 4.2	Application Modules	13
Table. 7.3	Test Cases	23 – 24
Table. 7.4	Results	24
Table. 8.1	Team Contributions	25

ABSTRACT

ShopEase is an e-commerce website developed as part of the Corizo Edutech Private Limited internship program. The project focuses on developing a web-based shopping platform that provides product browsing, user registration and login, shopping cart management, order placement, and administrative operations.

The frontend of the application is developed using HTML, CSS, and JavaScript to provide the user interface and interactive shopping experience. The backend is implemented using Python and the Flask framework, which provides API endpoints for user authentication, product management, cart operations, order processing, and administrative functions. The project includes database connectivity through PostgreSQL using the psycopg2 library, with the database connection configured through an environment variable. A local SQLite database file is also included in the project archive; its role in the application requires further verification.

The application includes separate customer-facing and administrator-facing modules. Customers can interact with product listings, manage their shopping cart, place orders, and view their orders. Administrative pages support product addition, dashboard access, customer viewing, and order management.

The project demonstrates the integration of frontend technologies, backend API development, and database connectivity in a web application. It also provides practical experience in organizing application modules, handling HTTP requests, managing data, and preparing a website for deployment.

This report describes the project's objectives, requirements, system design, technologies, implementation, testing, results, team contributions, conclusion, and future enhancements.

1. INTRODUCTION

1.1 Background

The growth of internet technology has transformed the way people purchase products and interact with businesses. E-commerce allows customers to browse products, compare available options, and place orders through online platforms without visiting physical stores. These systems provide convenient access to products and help businesses manage product information, customer details, and orders digitally.

Developing an e-commerce website requires the integration of several technologies, including frontend development, backend programming, database management, and user authentication. A well-designed system must provide an accessible interface while supporting reliable communication between its different components.

1.2 Project Overview

ShopEase is a web-based e-commerce application developed as part of the Corizo internship program. It provides a shopping interface where customers can browse available products and use shopping-related functionality.

The project also includes an administrative module for managing products, accessing customer information, and viewing orders. The frontend is built using HTML, CSS, and JavaScript, while the backend uses Python and the Flask framework. Database connectivity supports the storage and retrieval of application information.

The project is organized into separate directories for administrative pages, backend functionality, product images, and customer-facing pages. This structure helps separate the main components of the application and makes the codebase easier to maintain.

1.3 Problem Statement

Traditional shopping processes may require customers to visit physical stores to explore available products and place orders. Businesses also need effective ways to organize product information, maintain customer records, and manage orders.

ShopEase addresses these needs by providing an online shopping interface and administrative pages within a single web application. The system aims to simplify product browsing and order-related operations while demonstrating the practical application of modern web development technologies.

1.4 Scope of the Project

The scope of ShopEase includes the following functionalities:

- Customer module: User registration, login, product browsing, shopping cart operations, order placement, and order-related functionality.
- Product module: Displaying products with associated images and supporting product management.
- Administrative module: Admin login, dashboard access, product addition, customer viewing, and order viewing.
- Backend module: Flask API endpoints for application operations and communication between the frontend and backend.
- Database module: Database connectivity for application data management.
- Deployment: Hosting the frontend and backend through the project's configured deployment services, subject to deployment verification.

The project focuses on developing a functional e-commerce prototype. Payment gateway integration, advanced security features, inventory automation, and other enhancements should only be described as implemented if the source code and testing confirm them.

2. PROJECT OBJECTIVE

2.1 Primary Objective

The primary objective of ShopEase is to design and develop a web-based e-commerce application that provides essential online shopping features and administrative capabilities through an integrated frontend and backend system.

2.2 Specific Objectives

1. To create an interactive and user-friendly shopping interface.
2. To implement user registration and login pages.
3. To display product information and product images.
4. To provide shopping cart and order placement functionality.
5. To develop backend services using Python and Flask.
6. To connect the application to a database for storing and retrieving information.
7. To provide administrative pages for product, customer, and order management.
8. To organize the source code into manageable frontend and backend components.
9. To test the major application workflows and identify errors.
10. To gain practical experience in full-stack web development through collaborative internship work.

3. SYSTEM REQUIREMENTS AND ANALYSIS

3.1 Hardware Requirements

The following hardware configuration is suitable for developing and running the ShopEase web application.

Component	Minimum requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	500 MB of free space for project files, excluding optional development tools
Display	Standard monitor or laptop display
Network	Internet connection for accessing hosted services and online resources

Table 3.1: Hardware Requirements

3.2 Software Requirements

The project uses the following technologies and software components.

Technology	Purpose
HTML	Defines the structure of web pages
CSS	Controls layout, styling, and responsiveness
JavaScript	Adds interactive behavior to the frontend
Python	Implements backend application logic
Flask	Provides backend routes and API endpoints
PostgreSQL	Database technology supported by the backend configuration
psycopg2-binary	Enables Python connectivity to PostgreSQL
python-dotenv	Supports loading configuration from environment variables
Flask-CORS	Supports cross-origin requests where configured
Gunicorn	Provides a production-capable WSGI server
Web browser	Runs and displays the frontend
Git and GitHub	Support source-code version control and repository hosting

Table 3.2: Software Requirements

The project archive also contains an SQLite database file. Its role in the application should be confirmed before we describe it as the primary database.

3.3 Functional Requirements

Functional requirements describe the operations that the application is intended to perform.

3.3.1 User Module

- Allow users to access registration and login pages.
- Display the available products.
- Allow users to interact with the shopping cart.
- Support order placement.
- Provide order-related information to users.

3.3.2 Product Module

- Display product information and associated images.
- Provide administrative functionality for adding products.
- Support retrieving product information through backend services.

3.3.3 Admin Module

- Provide an administrator login page.
- Provide an administrative dashboard.
- Allow product addition.
- Display customer information.
- Display order information.
- Support order management operations where implemented.

3.3.4 Backend Module

- Process requests from the frontend.
- Provide API endpoints for application operations.
- Communicate with the configured database.
- Return appropriate responses to frontend requests.

Module	Functional Requirement	Description
User Module	User Registration	Users can create an account.
User Module	User Login	Registered users can log in.
Product Module	Product Browsing	Users can view available products and their details.
Shopping Cart	Cart Management	Users can add and remove products from their cart.
Order Module	Order Placement	Users can place orders for selected products.
Order Module	Order Viewing	Users can view their order information.

Admin Module	Product Management	Administrators can add and manage products.
Admin Module	Customer Management	Administrators can view customer information.
Admin Module	Order Management	Administrators can view and manage orders.

Table 3.3: Functional Requirements

3.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes expected from the application.

1. **Usability:** The interface should be understandable and straightforward to navigate.
2. **Performance:** Pages and API operations should respond within a reasonable time under normal usage.
3. **Maintainability:** The separation of frontend, backend, and administrative files should make future updates easier.
4. **Compatibility:** The website should work in modern web browsers.
5. **Reliability:** Core operations should handle valid requests and expected errors appropriately.
6. **Security:** Authentication, database access, and configuration should be handled securely.
7. **Scalability:** The application structure should permit future improvements as product and user requirements grow.

These are design expectations; we will evaluate actual behavior in the testing chapter.

Quality Attribute	Description
Usability	The website should provide a simple and user-friendly interface.
Performance	The website should respond to user requests efficiently.
Maintainability	The application should be easy to modify and maintain.
Compatibility	The website should work across commonly used web browsers and devices.
Reliability	The application should perform its functions consistently.
Security	User authentication and application data should be protected.
Scalability	The application should support future growth and additional features.

Table 3.4: Non-Functional Requirements

3.5 User Roles

ShopEase contains two main user-facing roles.

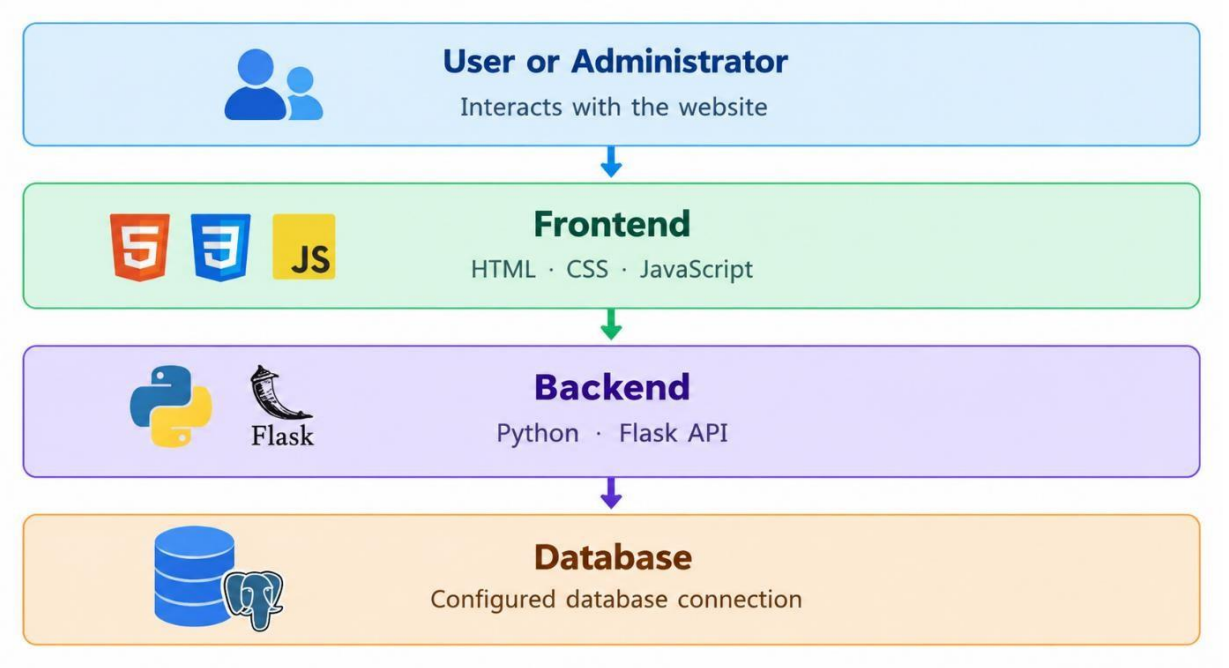
Role	Responsibilities
Customer	Browse products, use shopping functionality, and place orders
Administrator	Access administrative pages, add products, and view customer and order information

The precise permissions for each role will be documented after verifying the authentication and authorization logic in the source code.

3.6 System Analysis

The application follows a web-based architecture in which the frontend presents the user interface, the Flask backend handles application requests, and database services support data storage and retrieval.

The expected request flow is:



The diagram represents the general architecture; the final report should reflect the actual request and database flow confirmed from the source code.

4. SYSTEM DESIGN AND ARCHITECTURE

This chapter describes how ShopEase is organized, how its major modules interact, and how data moves through the application. We will follow the formal report style of your Corizo internship reference PDF.

4.1 System Architecture

ShopEase follows a web application architecture consisting of a frontend, a backend API, and database connectivity.

- **Frontend layer:** HTML, CSS, and JavaScript provide the customer-facing interface and administrative pages.
- **Backend layer:** Python and Flask handle application requests and backend operations.
- **Database layer:** The backend connects to a database for storing and retrieving application information.
- **Deployment layer:** The frontend and backend can be hosted separately, depending on the configured deployment services.

The project repository separates the pages/, admin/, and backend/ directories, helping organize the application's major components.

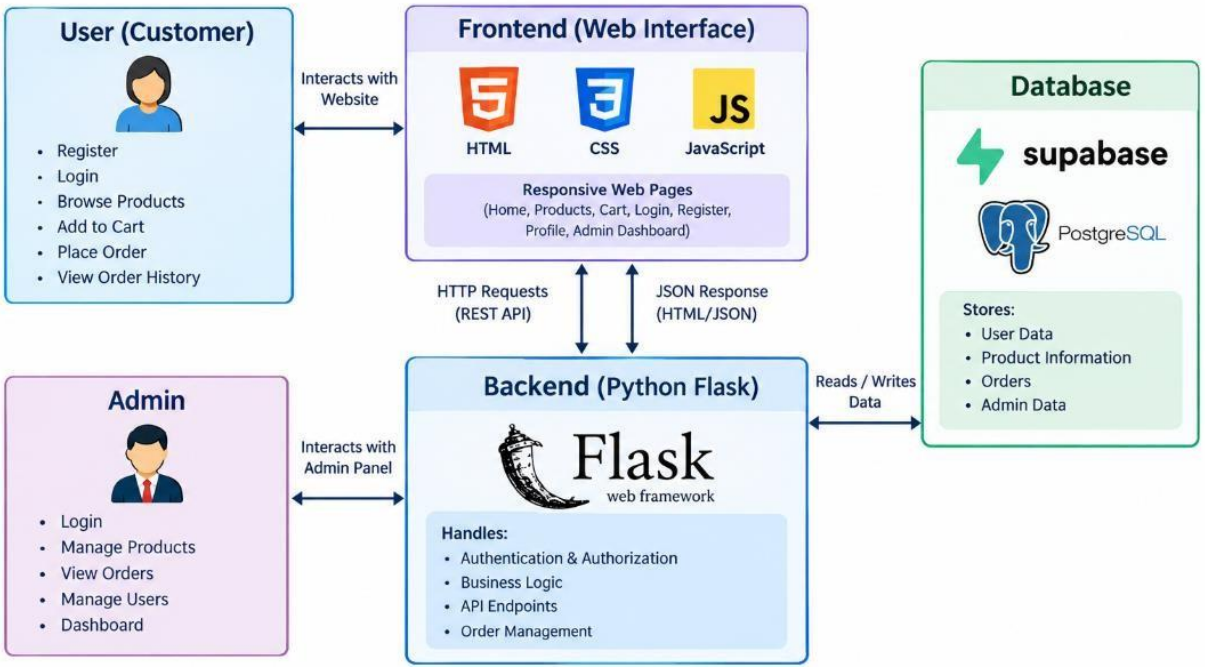


Figure 4.1: System Architecture Diagram

4.2 System Modules

ShopEase is divided into the following major modules based on the source-code structure.

Module	Main responsibilities
Customer interface	Registration, login, product browsing, cart and orders
Product management	Adding products and retrieving product information
Shopping cart	Adding, retrieving and removing cart items
Order management	Placing orders, retrieving orders and updating order status

Admin module	Admin login, dashboard, customer viewing and order viewing
Backend API	Processing requests and coordinating application operations
Database connectivity	Connecting the backend to the configured data store

Table 4.2: Application Modules

The Flask application contains API routes for these operations, including /api/products, /api/register, /api/login, /api/cart, /api/orders, /api/customers, and /api/admin/login.

4.3 Data Flow

The general data flow of the application is as follows:

1. The customer or administrator opens the appropriate webpage.
2. The user performs an action, such as logging in, adding a product, or placing an order.
3. JavaScript sends the relevant request to the Flask backend.
4. The backend processes the request and performs the required database operation.
5. The backend returns a response to the frontend.
6. The frontend displays the resulting information or an appropriate message.

This describes the general request-response workflow. Specific validation and error-handling behavior will be discussed in the implementation and testing chapters.

4.4 Database Design

The project archive contains backend/shopease.db, a SQLite database file, and the Flask source code includes a database connection function. The backend requirements also include psycopg2-binary, which supports PostgreSQL connectivity.

Therefore, we should not yet describe either SQLite or PostgreSQL as the sole production database without checking the connection configuration and actual database operations.

The application handles data related to:

- Users and authentication
- Products
- Shopping cart items
- Orders and order status
- Customer information

These are the main data categories supported by the application's routes. The final report's database tables, attributes, primary keys and relationships should be based on the actual schema.

4.5 Website Directory Structure

The following table summarizes the project files and their purposes.

File or directory	Purpose
index.html	Main website page
style.css	Website styling
script.js	Frontend JavaScript functionality
pages/	Customer-facing pages, including login, registration, products, cart and orders
admin/	Administrative pages
backend/app.py	Flask application and API routes
backend/requirements.txt	Python dependencies
images/	Product images used by the website
screenshots/	Project screenshots, if included in the archive

4.6 Architecture Summary

The ShopEase project combines frontend web technologies with a Flask backend and database connectivity. Its modular structure separates customer-facing pages from administrative pages and backend operations. This organization provides a foundation for maintaining the application and extending its functionality.

5. TECHNOLOGIES USED

This chapter describes the technologies and development tools used in the ShopEase e-commerce website. Each technology contributes to a particular part of the application's development and operation.

5.1 HTML (HyperText Markup Language)

HTML is used to define the structure and content of web pages. In ShopEase, HTML files provide the layout for the main website, customer-facing pages, and administrative interfaces. The project includes index.html and several HTML pages inside the admin/ and pages/ directories.

Applications in ShopEase:

- Creating the main website interface.
- Structuring product information and navigation.
- Providing registration and login forms.
- Creating administrative pages for product, customer, and order operations.

5.2 CSS (Cascading Style Sheets)

CSS is used to control the appearance and layout of the website. It helps create a consistent

interface across different pages and screen sizes.

The project contains a style.css file for styling the website.

Applications in ShopEase:

- Styling page layouts, forms, and buttons.
- Improving product presentation.
- Maintaining consistent colors, spacing, and typography.
- Supporting responsive layouts where implemented.

5.3 JavaScript

JavaScript adds interactive behavior to the frontend and enables communication with backend APIs.

ShopEase includes a script.js file for frontend functionality.

Applications in ShopEase:

- Handling user interactions.
- Sending requests to backend endpoints.
- Updating displayed information based on API responses.
- Supporting shopping cart and order-related operations.

5.4 Python

Python is the primary programming language used for the backend implementation. Its readable syntax and extensive libraries make it suitable for developing web applications and APIs.

Applications in ShopEase:

- Implementing backend logic.
- Processing requests from the frontend.
- Performing product, customer, and order operations.
- Coordinating database interactions.

5.5 Flask Framework

Flask is a Python web framework used to implement the backend application and its API endpoints.

The project contains backend/app.py, which defines the Flask application and its routes.

Applications in ShopEase:

- Handling HTTP requests.
- Providing product and authentication endpoints.
- Supporting cart and order operations.
- Exposing administrative operations.

- Returning responses to the frontend.

5.6 Database Technologies

Database connectivity supports storing and retrieving the information required by an e-commerce application.

The project dependencies include psycopg2-binary for PostgreSQL connectivity, while the archive also contains a SQLite database file.

The precise role of each database must be confirmed from the connection configuration and schema before the final report identifies the primary production database.

5.7 Flask-CORS

Flask-CORS is a Python extension that supports Cross-Origin Resource Sharing (CORS) configuration.

It can allow a frontend hosted on one origin to make permitted requests to a backend hosted on another origin. The actual CORS configuration in ShopEase determines which origins are allowed.

5.8 Python Dotenv

The python-dotenv package supports loading configuration values from a .env file into the application's environment.

This can be used to manage database connection settings and other configuration without embedding sensitive values directly in the source code.

Security consideration: passwords, API keys, and other secrets should not be included in the published project report or committed to a public GitHub repository.

5.9 Gunicorn

Gunicorn is a Python WSGI HTTP server that can run Flask applications in production environments.

The project includes Gunicorn in its backend dependencies. Its inclusion alone does not confirm which server is currently running the deployed application.

5.10 Git and GitHub

Git is a version-control system used to track changes to project files. GitHub provides a platform for hosting Git repositories and collaborating on software projects.

ShopEase has a public GitHub repository:

https://github.com/sharathjakki1316/Ecommerce_Website?utm_source=chatgpt.com

Git and GitHub support source-code organization, version history, collaboration, and project sharing.

5.11 Summary of Technologies

Technology	Main purpose
HTML	Webpage structure
CSS	Styling and layout
JavaScript	Frontend interactivity
Python	Backend programming
Flask	Web framework and API development
PostgreSQL connectivity	Database access through psycopg2
SQLite	Database file included in the project archive; role to be verified
Flask-CORS	Cross-origin request configuration
Python Dotenv	Environment-based configuration
Gunicorn	WSGI server option
Git and GitHub	Version control and repository hosting

6. SYSTEM IMPLEMENTATION

This chapter explains how the ShopEase e-commerce application is organized and how its main modules support online shopping and administrative operations. The descriptions are based on the project structure and backend routes identified so far. Actual behavior and successful execution will be documented separately in the testing chapter.

6.1 Frontend Implementation

The frontend is developed using HTML, CSS, and JavaScript. It provides the interface through which customers and administrators interact with the application.

The project contains index.html as the main webpage, style.css for styling, script.js for frontend behavior, and a pages/ directory for customer-facing pages.

The frontend is responsible for presenting product information, collecting user input, handling interactions, and communicating with the backend API where required.

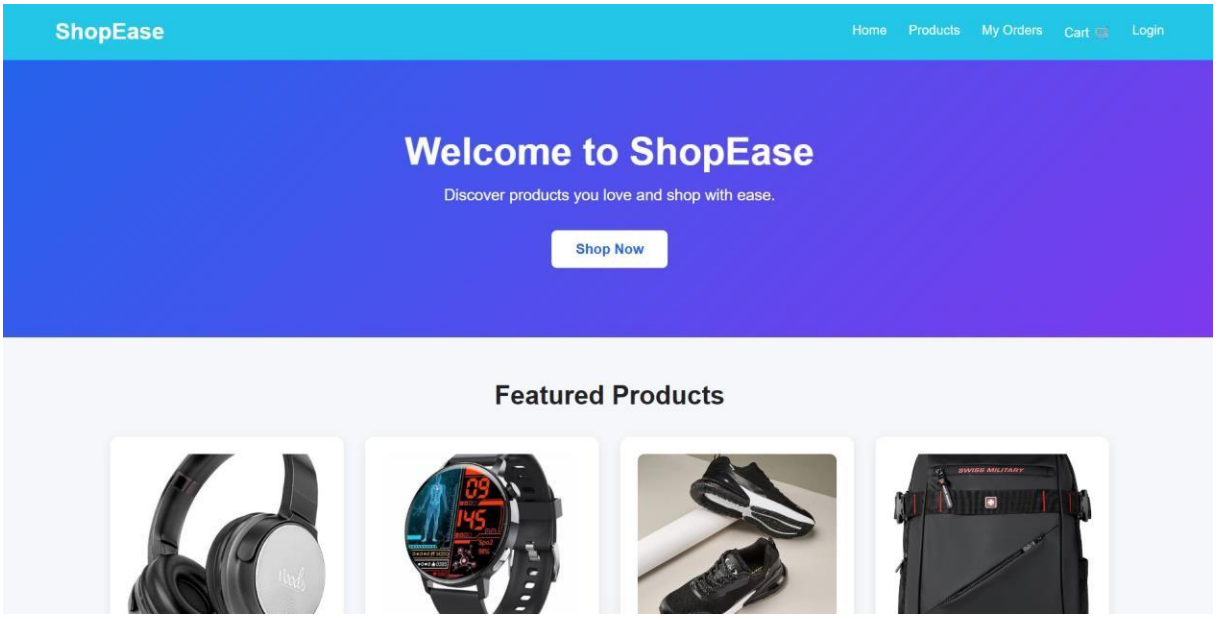


Figure 6.1: ShopEase Home Page

6.2 User Registration and Login Module

The registration and login pages provide the interface for customers to create accounts and access the application.

The backend includes the /api/register and /api/login routes for authentication-related operations.

The registration module is intended to collect the required user details, while the login module submits authentication information for backend processing. The exact validation rules, password-storage mechanism, and authentication behavior must be confirmed from the implementation and testing results.

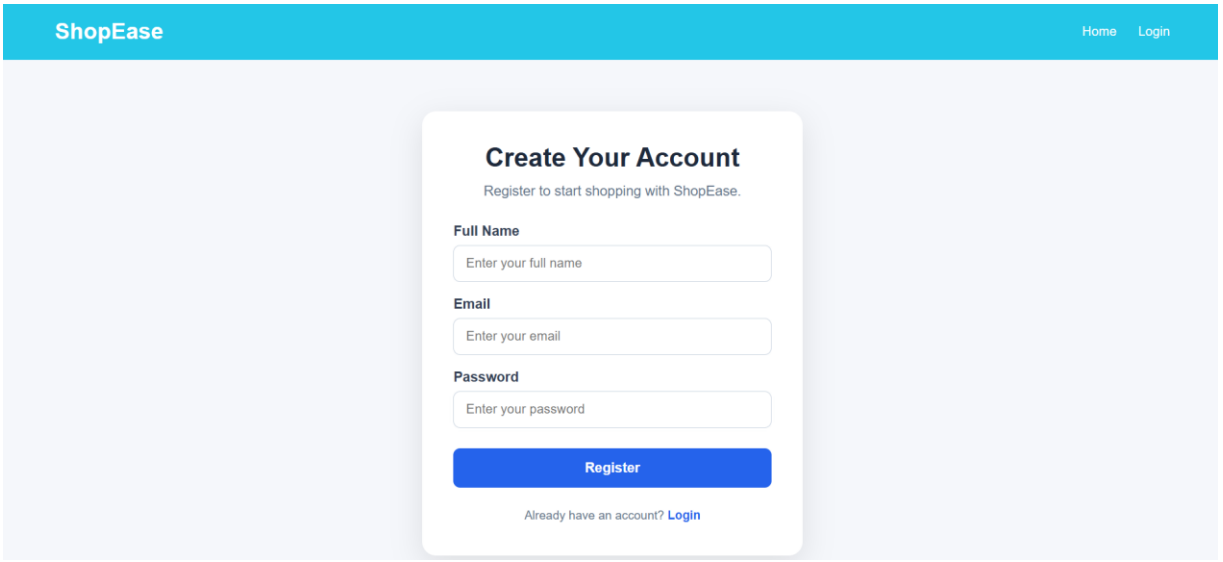


Figure 6.2.1: User Registration Page

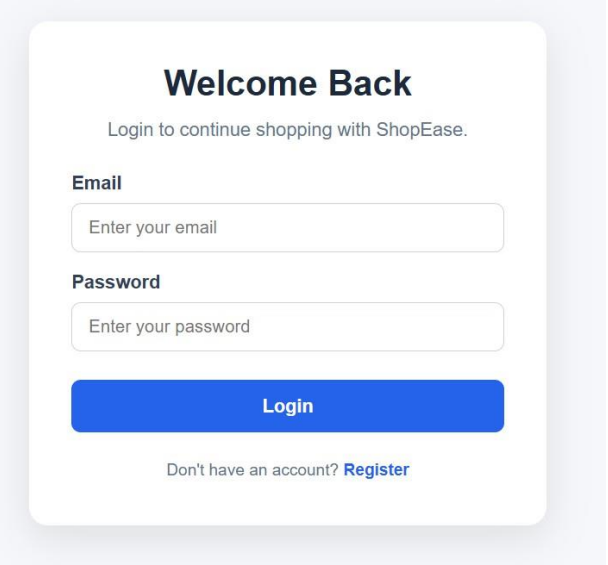


Figure 6.2.2: User Login Page

6.3 Product Browsing Module

The product browsing module allows customers to explore the products available on the website.

The project includes product images such as backpacks, headphones, laptops, shoes, smartwatches, and speakers. These images are stored in the images/ directory.

The backend includes the /api/products endpoint for product-related requests.

The module is intended to provide product information to the frontend so that customers can review available items before making a purchase.

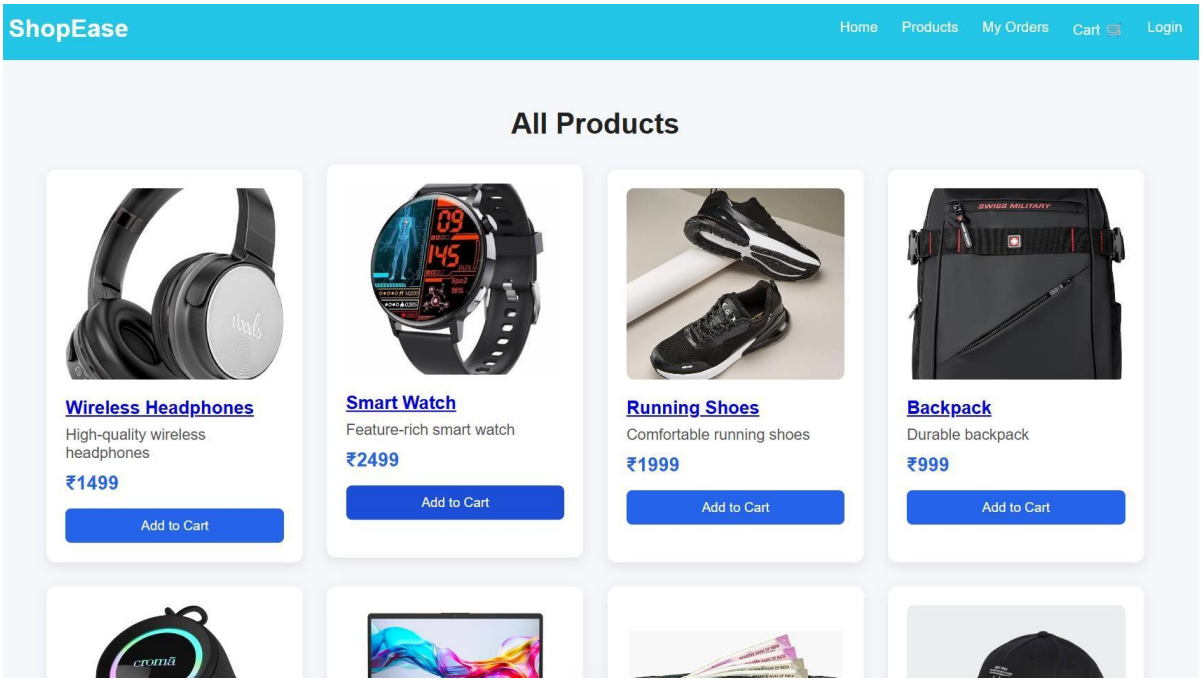


Figure 6.3: Product Listing Page

6.4 Shopping Cart Module

The shopping cart module supports the process of selecting products for purchase.

The project includes a /api/cart backend route for cart-related operations.

The module is intended to support cart operations such as adding products, retrieving cart contents, and removing items. The exact behavior, including quantity updates and cart persistence, should be confirmed through source-code inspection and testing.

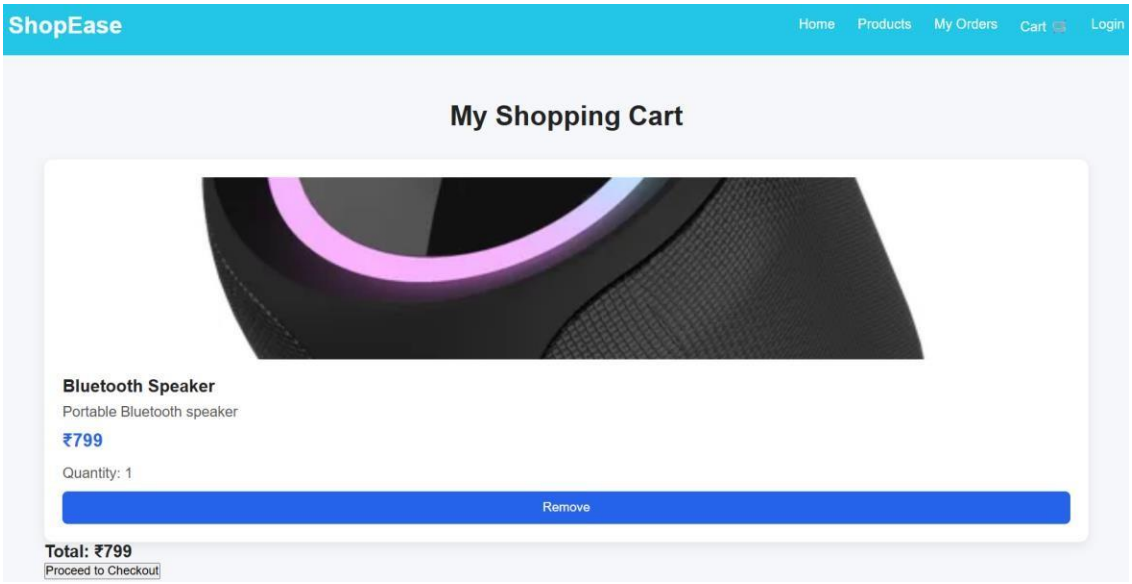


Figure 6.4: Shopping Cart Page

6.5 Order Management Module

The order management module supports order-related operations in ShopEase.

The backend includes the /api/orders endpoint, and the administrative directory contains a view-orders.html page.

The module is intended to connect customer order placement with administrative order viewing. Order status updates and order-history behavior should be documented according to the implemented routes and verified results.

6.6 Admin Login Module

The administrator login page is located at admin/admin-login.html.

The backend includes an /api/admin/login endpoint for administrative authentication.

The login interface provides the entry point to the administrative module. The implementation's access-control checks should be tested to ensure that administrative functions cannot be accessed by unauthorized users.

6.7 Admin Dashboard Module

The administrative dashboard is located at admin/admin-dashboard.html.

It serves as the main interface for accessing administrative functionality. The project also contains separate pages for product addition, customer viewing, and order viewing.

The dashboard is intended to help administrators navigate these functions from a centralized interface.

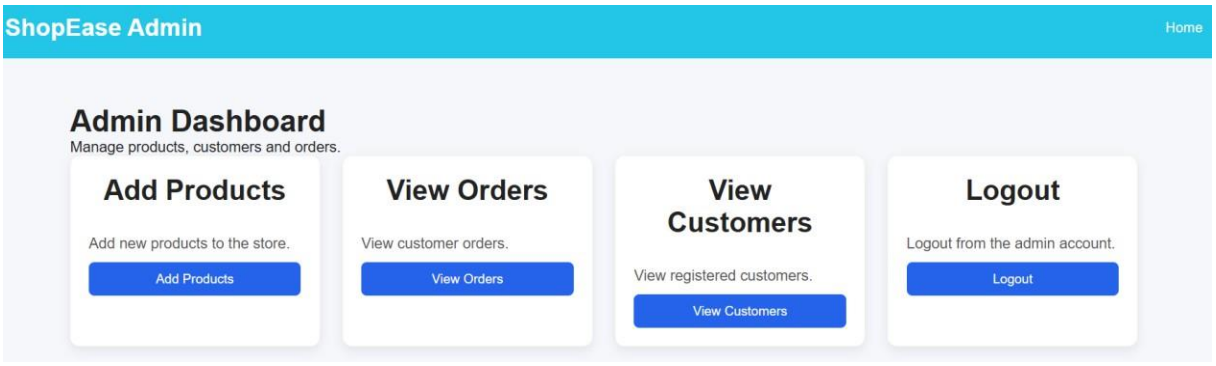


Figure 6.7: Admin Dashboard

6.8 Product Management Module

The product addition page is located at admin/add-product.html.

This module is intended to let administrators enter product information and submit it for backend processing. The backend's product-management logic and database operations determine which fields are stored and how newly added products appear on the customer-facing website.

The report's final version should specify the exact product fields and any image-upload behavior after these details have been verified.

6.9 Customer Management Module

The administrative customer page is located at admin/view-customers.html.

The backend includes an /api/customers endpoint for customer-related requests.

This module is intended to provide administrators with access to customer information. The final report should specify the information displayed and the authorization restrictions after the page and API have been tested.

6.10 Backend API Implementation

The backend is implemented in Python using Flask, with the main application file located at backend/app.py.

The identified routes include:

API endpoint	Purpose
/api/products	Product-related operations
/api/register	User registration
/api/login	User login
/api/cart	Shopping cart operations

/api/orders	Order-related operations
/api/customers	Customer-related operations
/api/admin/login	Administrator login

These endpoint names indicate their intended responsibilities. HTTP methods, request parameters, validation, and response formats should be included only after verifying the corresponding route implementations.

6.11 Summary

The ShopEase implementation is organized into customer-facing pages, administrative pages, product assets, and a Flask backend. This modular structure separates the presentation layer from server-side processing and provides a foundation for integrating product browsing, authentication, cart management, and order operations.

7. TESTING AND RESULTS

Software testing is an important stage in the development of an e-commerce application. It helps determine whether the implemented features behave as expected and identifies defects that may affect usability, reliability, or security.

In the ShopEase project, testing should cover the customer-facing interface, backend API, database operations, and administrative module.

7.1 Objectives of Testing

The main objectives of testing ShopEase are:

1. To verify that the website pages load correctly.
2. To check registration and login functionality.
3. To verify product display and retrieval.
4. To test shopping cart operations.
5. To verify order placement and order retrieval.
6. To check administrative login and access to management pages.
7. To verify product addition and customer/order viewing.
8. To identify errors in frontend-backend communication.
9. To evaluate the application's behavior when invalid input or failed requests occur.

7.2 Testing Methodology

The following testing methods are relevant to ShopEase.

- Functional testing: Verifies that individual features perform their intended operations.
- Integration testing: Checks communication between the frontend, Flask backend, and database.

- Interface testing: Examines page layout, navigation, forms, and displayed messages.
- Negative testing: Checks how the application responds to invalid input and unsuccessful requests.
- Deployment testing: Verifies that the deployed frontend can communicate with the deployed backend.

7.3 Test Cases

The following table provides the test plan for the main application features. These are proposed tests, not recorded test results.

Test ID	Test scenario	Expected result
TC-01	Open the main website	Homepage loads correctly
TC-02	Open the registration page	Registration form is displayed
TC-03	Register with valid details	Account is created and an appropriate response is shown
TC-04	Log in with valid credentials	Authentication succeeds
TC-05	Log in with invalid credentials	Access is denied and an appropriate error is shown
TC-06	Retrieve product listings	Available products are displayed
TC-07	Add a product to the cart	Product is added to the cart
TC-08	Remove a cart item	Item is removed from the cart
TC-09	Place an order	Order is processed and confirmation is shown
TC-10	Retrieve customer orders	Relevant order information is displayed
TC-11	Open the admin login page	Administrator login interface is displayed
TC-12	Add a product through the admin page	Product is stored and appears in the appropriate listing
TC-13	View customers in the admin module	Authorized administrator can view customer information
TC-14	View orders in the admin module	Authorized administrator can view order information
TC-15	Submit invalid form data	Invalid input is handled appropriately

Table 7.3: Test Cases

7.4 Test Results

Actual results must be recorded after executing the tests. Use the following table to document

the outcomes.

Test category	Status
Homepage and interface	Passed
Registration and login	Passed
Product browsing	Passed
Shopping cart	Passed
Order operations	Passed
Admin authentication	Passed
Product management	Passed
Customer and order viewing	Passed
Deployment and API connectivity	Passed

Table 7.4: Test Results

Important: We should replace these pending statuses with actual outcomes only after testing the application. Screenshots of successful operations can then be added as evidence in the final report.

7.5 Result Analysis

The ShopEase project provides separate frontend and backend components, customer-facing pages, administrative interfaces, and product-image assets. These components establish the structure needed for an e-commerce application.

The final evaluation will depend on the results of functional and integration testing. Particular attention should be given to authentication, database operations, cart persistence, order creation, and administrator authorization.

7.6 Limitations of the Current Evaluation

At this stage, the report does not contain verified test execution logs or measured performance results. Therefore, claims about complete reliability, security, or successful execution of every feature would be premature.

The testing phase should produce actual results, screenshots, and a record of any defects identified and corrected.

8. TEAM CONTRIBUTIONS

The ShopEase project was developed as a team activity during the internship at Corizo Edutech Private Limited. The project involves frontend development, backend implementation, database operations, testing, and documentation.

The following table provides a proposed distribution of responsibilities for the five team members. Please confirm that these assignments reflect the work each member actually completed before submitting the report.

8.1 Team Member Responsibilities

S. No.	Team member	Proposed responsibilities
1	VENKATA SHARTH KUMAR JAKKI	Project coordination, frontend integration, and overall project documentation
2	SHRATI TIRUMALESHWAR HEDGE	User interface development and registration/login pages
3	SVVVS. BHASKAR	Flask backend development and API integration
4	PRAADEEP S	Database operations and product/order management
5	DHANVI S	Testing, documentation support, and deployment verification

Table 8.1: Team contributions

8.2 Collaborative Development

The team worked toward building an e-commerce website that combines a customer-facing interface with backend services and administrative functionality.

The major collaborative activities include:

- Planning the application structure and identifying the required modules.
- Developing and integrating website pages.
- Connecting frontend operations with backend API endpoints.
- Managing product, customer, cart, and order-related functionality.
- Identifying implementation issues and testing application behavior.
- Maintaining project files and documenting the development process.

8.3 Importance of Teamwork

Teamwork helped divide the project into manageable tasks and allowed different components to be developed and integrated. Coordination between frontend and backend development was particularly important because user actions on the website depend on appropriate API responses and database operations.

The internship project also provided practical exposure to software development workflows, collaboration, problem-solving, and project documentation.

9. CONCLUSION

The ShopEase project focuses on the development of a web-based e-commerce application with customer-facing pages and administrative interfaces. The project brings together frontend technologies, backend services, and database-related operations to establish the structure of an online shopping system.

The frontend is designed using HTML, CSS, and JavaScript, while Python and Flask are used for backend development. The project structure includes functionality related to user registration, login, product browsing, shopping cart operations, order management, and administrative activities.

The development process provides practical experience in organizing a web application into separate components, connecting frontend pages with backend APIs, and managing project files using version control. It also demonstrates the importance of requirements analysis, system design, implementation, testing, and team collaboration in software development.

The project should be evaluated through functional and integration testing to confirm that the implemented features work as expected. Database connectivity, authentication, order processing, and deployment configuration should also be verified before the application is presented as a fully tested system.

Overall, ShopEase serves as an internship project for applying web development concepts to an e-commerce use case and gaining practical experience in collaborative software development.

10. FUTURE ENHANCEMENTS

The ShopEase e-commerce website can be improved in several ways to provide a better shopping experience, strengthen security, and support future growth. The following enhancements are proposed possibilities and should not be described as completed features unless they have already been implemented.

10.1 Online Payment Integration

A secure payment gateway could be integrated to allow customers to pay online using supported payment methods such as UPI, debit cards, and credit cards. Payment verification and transaction status handling would be important parts of this enhancement.

10.2 Product Search and Filtering

A product search feature could help customers find products using names or keywords. Additional filters could be introduced for product categories, price ranges, and other relevant attributes.

10.3 Improved User Authentication and Security

The application could be strengthened by implementing secure password hashing, input

validation, protected administrative routes, secure session handling, and appropriate access controls. Sensitive configuration values should be stored outside publicly accessible source code.

10.4 Order Tracking

Customers could be provided with an order history and status-tracking page. Administrators could update order statuses, allowing customers to follow the progress of their purchases.

10.5 Product Reviews and Ratings

A review and rating system could allow customers to share feedback about purchased products. This could help other customers make more informed purchasing decisions.

10.6 Responsive User Interface

The interface could be further optimized for mobile phones, tablets, and different screen sizes. Responsive layouts, accessible forms, and clear navigation would improve usability across devices.

10.7 Inventory Management

An inventory management module could track product quantities and update stock levels when orders are processed. It could also notify administrators when products are running low.

10.8 Notifications

Email or other appropriate notifications could be added for registration, order confirmation, order-status changes, and other important activities.

10.9 Performance and Scalability

Performance could be improved through optimized database queries, image optimization, caching where appropriate, and systematic error handling. Further improvements would depend on actual usage requirements and performance testing.

10.10 Reporting and Analytics

An administrative analytics dashboard could display information such as order counts, sales summaries, popular products, and customer activity. These reports could support better business decisions.

10.11 Summary

These enhancements provide possible directions for the future development of ShopEase. Their implementation would depend on project requirements, available resources, security considerations, and feedback from users.

11. REFERENCES

The following references identify the principal technologies and project resources relevant to

ShopEase.

1. Corizo Edutech Private Limited — Internship and project development context.
2. ShopEase Project Repository — Source code and project files.

https://github.com/sharathjakki1316/Ecommerce_Website?utm_source=chatgpt.com

3. HTML — MDN Web Docs — Reference for webpage structure and markup.

https://developer.mozilla.org/en-US/docs/Web/HTML?utm_source=chatgpt.com

4. CSS — MDN Web Docs — Reference for styling and layout.

https://developer.mozilla.org/en-US/docs/Web/CSS?utm_source=chatgpt.com

5. JavaScript — MDN Web Docs — Reference for client-side programming.

https://developer.mozilla.org/en-US/docs/Web/JavaScript?utm_source=chatgpt.com

6. Python Documentation — Reference for Python programming.

https://docs.python.org/3/?utm_source=chatgpt.com

7. Flask Documentation — Reference for developing web applications and API routes.

https://flask.palletsprojects.com/en/stable/?utm_source=chatgpt.com

8. PostgreSQL Documentation — Reference for relational database concepts and SQL operations, if PostgreSQL is used in the deployed configuration.

https://www.postgresql.org/docs/?utm_source=chatgpt.com

9. Git Documentation — Reference for version control and collaboration.

https://git-scm.com/docs?utm_source=chatgpt.com